

Serverless Lecture Notes - Part 1

Today we will talk about serverless computing and why it changed how teams build software. In the traditional model, a team rents servers, installs an operating system, and keeps the machines patched and running all day, even when nobody is using the application.

Serverless computing removes that burden. The cloud provider runs the servers, and the developer only supplies code and configuration. You pay for the time your code actually runs, measured in milliseconds, instead of paying for idle machines.

The best known serverless compute service is a function service. A function is a small piece of code that starts when an event arrives, does one job, and stops. An event can be a file upload, a web request, a message in a queue, or a timer.

Serverless Lecture Notes - Part 2

Object storage is the second building block. A storage bucket holds files of any size and keeps several copies for durability. Buckets are private by default, and access is granted through policies. A web browser can upload straight into a bucket using a temporary signed link.

The third building block is a managed database. A key value database stores items under a partition key and a sort key. Because each user owns a partition, listing the documents of one user is a fast query that never touches the data of other users.

Authentication is the fourth building block. A user pool stores accounts, checks passwords, and issues signed tokens. When a request arrives at the application programming interface, the gateway verifies the token before any of our code runs, so unauthenticated calls are rejected early.

Serverless Lecture Notes - Part 3

Workflows connect these pieces. A state machine describes the steps of a job, such as split the recording, transcribe each part, combine the text, summarize, and index. The service records every step, retries temporary failures, and marks the whole job as failed when a step cannot succeed.

Retries deserve a closer look. Temporary problems, like a busy model endpoint or a network timeout, usually disappear after a short wait. A good workflow waits a little longer before every new attempt. This pattern is called exponential backoff, and it protects both our application and the service we call.

Permanent problems are different. If a file is damaged or a document contains only scanned images, trying again will never help. The correct response is to stop quickly, store a readable error message, and show it to the user, so that no item stays stuck in a processing state forever.

Serverless Lecture Notes - Part 4

Security in the cloud follows the principle of least privilege. Every function gets its own role, and each role lists only the actions that function needs, on only the resources it touches. If one function is compromised, the damage stays small.

Secrets such as API keys must never appear in source code or in the template of an infrastructure stack. A parameter store keeps them encrypted. The function reads the value at run time, using a permission that allows reading that single parameter and nothing else.

Infrastructure as code means that the whole environment is described in text files. A single command can create the storage, the database, the functions, and the alarms. The same command can delete them again, which makes experiments cheap and repeatable.

Serverless Lecture Notes - Part 5

Continuous integration runs checks on every proposed change. The pipeline lints the code, runs the unit tests, validates the templates, and scans for secrets. Only a change that passes every check may be merged. A merge to the main branch can then deploy automatically to a shared environment.

Monitoring closes the loop. A dashboard shows function errors, workflow failures, processing time, and latency of the interface. An alarm sends an email to the team when a run fails, so problems are noticed in minutes instead of days.

Cost is the last topic. Serverless services have generous free tiers, but the speech and language models are billed per use. A cost per document estimate adds the price of the audio minutes, the tokens, and the small charges for storage, orchestration, and function time.

Serverless Lecture Notes - Part 6

To summarize, a serverless application combines storage, functions, a database, authentication, and a workflow engine. Each part is managed, scales on its own, and costs almost nothing when idle. The engineering effort moves from running machines to designing events, permissions, and failure handling.

Let us review the key terms once more. An event triggers a function. A bucket stores files. A table stores metadata. A token proves identity. A state machine coordinates steps. A role limits permissions. A dashboard shows health. An alarm tells people when something breaks.

Finally, remember the limits of this design. Audio must be in a supported format, documents must contain real text, and keyword search only finds answers that share words with the question. Knowing these limits, and stating them clearly, is part of good engineering.

Serverless Lecture Notes - Part 7

Today we will talk about serverless computing and why it changed how teams build software. In the traditional model, a team rents servers, installs an operating system, and keeps the machines patched and running all day, even when nobody is using the application.

Serverless computing removes that burden. The cloud provider runs the servers, and the developer only supplies code and configuration. You pay for the time your code actually runs, measured in milliseconds, instead of paying for idle machines.

The best known serverless compute service is a function service. A function is a small piece of code that starts when an event arrives, does one job, and stops. An event can be a file upload, a web request, a message in a queue, or a timer.

Serverless Lecture Notes - Part 8

Object storage is the second building block. A storage bucket holds files of any size and keeps several copies for durability. Buckets are private by default, and access is granted through policies. A web browser can upload straight into a bucket using a temporary signed link.

The third building block is a managed database. A key value database stores items under a partition key and a sort key. Because each user owns a partition, listing the documents of one user is a fast query that never touches the data of other users.

Authentication is the fourth building block. A user pool stores accounts, checks passwords, and issues signed tokens. When a request arrives at the application programming interface, the gateway verifies the token before any of our code runs, so unauthenticated calls are rejected early.

Serverless Lecture Notes - Part 9

Workflows connect these pieces. A state machine describes the steps of a job, such as split the recording, transcribe each part, combine the text, summarize, and index. The service records every step, retries temporary failures, and marks the whole job as failed when a step cannot succeed.

Retries deserve a closer look. Temporary problems, like a busy model endpoint or a network timeout, usually disappear after a short wait. A good workflow waits a little longer before every new attempt. This pattern is called exponential backoff, and it protects both our application and the service we call.

Permanent problems are different. If a file is damaged or a document contains only scanned images, trying again will never help. The correct response is to stop quickly, store a readable error message, and show it to the user, so that no item stays stuck in a processing state forever.

Serverless Lecture Notes - Part 10

Security in the cloud follows the principle of least privilege. Every function gets its own role, and each role lists only the actions that function needs, on only the resources it touches. If one function is compromised, the damage stays small.

Secrets such as API keys must never appear in source code or in the template of an infrastructure stack. A parameter store keeps them encrypted. The function reads the value at run time, using a permission that allows reading that single parameter and nothing else.

Infrastructure as code means that the whole environment is described in text files. A single command can create the storage, the database, the functions, and the alarms. The same command can delete them again, which makes experiments cheap and repeatable.